

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
— o0o —



Bài tập lớn kỳ 2025.2

**Thiết kế nền tảng trò chơi nhúng trên STM32F429
với engine đồ họa 2D/3D dựa trên driver ILI9341**

Học phần: Hệ nhúng
Mã học phần: IT4210

Giảng viên hướng dẫn: TS. Đỗ Công Thuận

Sinh viên thực hiện: Nguyễn Quý Đức 20235682

Trần Đăng Sinh 20235821

Nguyễn Hồng Dương 20235693

Bùi Quang Phương 20235809

Mã lớp: 166152

Hà Nội - 2026

Tóm tắt

Báo cáo trình bày việc thiết kế một nền tảng trò chơi nhúng chạy trực tiếp trên vi điều khiển STM32F429ZIT6. Hệ thống tích hợp hai trò chơi có đặc điểm xử lý khác nhau: Stack 3D sử dụng bộ dựng hình ba chiều viết bằng C, còn Tetris sử dụng lưới hai chiều và cơ chế va chạm theo ô. Hình ảnh được xuất lên màn hình ILI9341 độ phân giải 320×240 qua SPI5. Người dùng điều khiển bằng nút PA0 hoặc bàn phím máy tính thông qua cầu nối Bluetooth Classic trên ESP32 và đường UART2 có DMA vòng.

Đóng góp kỹ thuật chính của đề tài là bộ dựng hình phần mềm phù hợp với giới hạn bộ nhớ của vi điều khiển. Pipeline thực hiện biến đổi World–View–Projection, loại mặt khuất, chiếu sáng phẳng, raster hóa scanline và kiểm tra độ sâu. Thay vì cấp phát framebuffer màu và độ sâu cho toàn màn hình, hệ thống xử lý theo hai dải, mỗi dải cao 120 dòng. Hai bộ đệm theo dải chiếm 153600 byte, bằng một nửa phương án toàn khung hình.

Mã nguồn đã bao gồm trình quản lý trạng thái, lớp ánh xạ đầu vào, hai trò chơi, driver màn hình và bộ dựng hình 2D/3D. ESP32 được cấp nguồn từ cổng USB OTG, còn dữ liệu điều khiển đi một chiều từ ESP32 đến STM32 qua UART2. Báo cáo đi kèm các hình ảnh thực nghiệm của cả hai trò chơi Stack 3D và Tetris chạy thực tế trên phần cứng nhằm xác minh hoạt động đúng đắn của hệ thống. **Từ khóa:** STM32F429, hệ thống nhúng, đồ họa 3D phần mềm, Z-buffer, ILI9341, Tetris, Bluetooth SPP.

Mục lục

Tóm tắt	i
1 Tổng quan đề tài	1
1.1 Bối cảnh và động lực	1
1.2 Bài toán cần giải quyết	1
1.3 Mục tiêu	1
1.4 Phạm vi và giới hạn	2
1.5 Yêu cầu hệ thống	2
1.5.1 Yêu cầu chức năng	2
1.5.2 Yêu cầu phi chức năng	3
2 Cơ sở kỹ thuật và kiến trúc hệ thống	4
2.1 Nền tảng phần cứng	4
2.2 Đường điều khiển không dây	4
2.3 Kiến trúc tổng thể	5
2.4 Tổ chức phần mềm	5
2.5 Luồng thực thi	6
3 Thiết kế và hiện thực	7
3.1 Quản lý trạng thái ứng dụng	7
3.2 Chuẩn hóa đầu vào	7
3.3 Pipeline dựng hình 3D	8
3.4 Tối ưu bộ nhớ theo chunk	9
3.5 Thiết kế Stack 3D	10
3.6 Thiết kế Tetris	10
3.7 Tích hợp hiển thị và giao diện	11
4 Cài đặt, kết quả và đánh giá	12
4.1 Môi trường phát triển và cách cài đặt	12
4.2 Các mô-đun cài đặt chính	12
4.3 Phân công và mức độ đóng góp	12
4.4 Kết quả và bằng chứng demo	13
4.5 Đánh giá	14
4.6 AI-assisted disclaimer	14
5 Kết luận và hướng phát triển	15
5.1 Kết quả đạt được	15
5.2 Đóng góp của đề tài	15
5.3 Hạn chế và hướng phát triển	15
Tài liệu tham khảo	16

CHƯƠNG 1

Tổng quan đề tài

1.1 Bối cảnh và động lực

Các hệ thống nhúng hiện đại có đủ năng lực để thực hiện giao diện đồ họa và trò chơi tương tác mà không cần hệ điều hành. Đây là bài toán phù hợp để khảo sát đồng thời nhiều nội dung của kỹ thuật máy tính: cấu hình ngoại vi, quản lý bộ nhớ, xử lý thời gian thực, mô hình trạng thái, giao tiếp không dây và thuật toán đồ họa. STM32F429ZIT6 sử dụng lõi Arm Cortex-M4 có FPU, tích hợp 2 MB Flash và tổng cộng 256 KB SRAM theo tài liệu của nhà sản xuất [1]. Cấu hình này đủ để xây dựng một thiết bị chơi game nhỏ, nhưng vẫn buộc người thiết kế phải kiểm soát chặt bộ đệm đồ họa và chi phí tính toán.

Đề tài phát triển *STM32 Retro Play*, một nền tảng gồm menu chọn trò chơi, Stack 3D và Tetris. Stack 3D đặt trọng tâm vào bộ dựng hình ba chiều viết hoàn toàn bằng C; Tetris đại diện cho trò chơi lưới hai chiều có luật va chạm, xoay khối và tính điểm. Hệ thống hiển thị trên LCD ILI9341 320×240 và nhận lệnh từ nút vật lý hoặc cầu nối Bluetooth qua ESP32. Toàn bộ phân tích trong báo cáo bám theo mã nguồn của dự án.

1.2 Bài toán cần giải quyết

Một khung hình đầy đủ ở định dạng RGB565 cần $320 \times 240 \times 2 = 153600$ byte. Nếu bổ sung Z-buffer 16 bit cho dựng hình 3D, riêng hai framebuffer đã cần 307200 byte, lớn hơn vùng RAM 192 KiB mà linker hiện dành cho dữ liệu thông thường. Bên cạnh giới hạn bộ nhớ, hệ thống còn phải xử lý luật của hai trò chơi, cập nhật theo thời gian, dựng giao diện và truyền số lượng lớn điểm ảnh qua SPI mà không dùng hệ điều hành.

Bài toán được chia thành bốn nhóm:

- xây dựng kiến trúc mô-đun để menu và các trò chơi dùng chung đầu vào, hiển thị và vòng lặp chính;
- hiện thực bộ dựng hình 3D có phép chiếu phối cảnh, loại mặt khuất, chiếu sáng phẳng và kiểm tra độ sâu;
- hiện thực luật Stack 3D và Tetris với trạng thái trò chơi rõ ràng;
- tích hợp đường điều khiển từ PC qua Bluetooth Classic, ESP32 và UART2 trên STM32.

1.3 Mục tiêu

Mục tiêu tổng quát là tạo một nguyên mẫu nền tảng trò chơi nhúng chạy trực tiếp trên STM32F429 và có cấu trúc đủ rõ để bổ sung trò chơi mới. Các mục tiêu cụ thể gồm:

1. Hiển thị menu và hai trò chơi trên LCD ILI9341 ở chế độ ngang;
2. Ánh xạ ký tự đầu vào thành hành động logic dùng chung cho menu, Stack 3D và Tetris;
3. Duy trì luật chơi, điểm số, màn hình kết thúc và thao tác khởi động lại;
4. Ghi nhận trung thực phần đã hiện thực, phần mới cấu hình và phần chưa có số đo thực nghiệm.

1.4 Phạm vi và giới hạn

Phạm vi báo cáo bao gồm mã ứng dụng trong Core/, cầu nối trong Others/, cấu hình STM32CubeMX và các thư viện STM32CubeF4 được repository sử dụng.

1.5 Yêu cầu hệ thống

Bảng 1.1 tổng hợp các yêu cầu có thể đối chiếu trực tiếp với mã nguồn và bằng chứng trong repository.

Bảng 1.1. Yêu cầu chính của nền tảng

Nhóm	Yêu cầu	Tiêu chí đối chiếu
Hiển thị	Xuất giao diện 320×240 ở định dạng RGB565	Có driver ILI9341 và truyền khối điểm ảnh qua SPI5
Đầu vào	Dùng chung hành động lên, xuống, trái, phải và chọn	Mọi ký tự đi qua <code>InputManager_MapCharacter</code>
Stack 3D	Cắt phần thừa, tạo khối rơi và tăng tốc theo điểm	Kích thước khối mới bằng phần giao nhau với khối đỉnh
Tetris	Hỗ trợ bảy tetromino, xoay, xóa dòng, điểm và cấp độ	Lưới 20×10 , kiểm tra va chạm trước mọi dịch chuyển
Bộ nhớ	Không cấp phát đồng thời framebuffer màu và độ sâu toàn màn hình	Dựng theo hai chunk, mỗi chunk cao 120 dòng
Tính đúng đắn	Không tuyên bố kết quả chưa có bằng chứng	Chỉ số chưa đo và ảnh thiếu phải có nhãn chờ

1.5.1 Yêu cầu chức năng

Hệ thống phải khởi tạo được màn hình ILI9341 ở chế độ ngang, hiển thị menu lựa chọn và chạy được hai trò chơi Stack 3D và Tetris. Người dùng phải điều hướng được menu, thực hiện các thao tác đặc trưng của từng trò chơi, xem điểm số, nhận biết trạng thái game over, khởi động lại trò chơi hoặc quay về menu. Lệnh điều khiển được tiếp nhận từ nút PA0 và từ bàn phím PC thông qua cầu nối Bluetooth Classic trên ESP32. Dữ liệu từ ESP32 được chuyển tới STM32 bằng UART2 và chuẩn hóa thành tập hành động dùng chung trước khi chuyển cho menu hoặc game.

1.5.2 Yêu cầu phi chức năng

Nguyên mẫu phải hoạt động ổn định trên STM32F429ZIT6 mà không cần RTOS, không vượt vùng RAM do linker cấp cho dữ liệu thông thường, và không cấp phát động trong vòng lặp dựng hình. Thiết kế phần mềm cần có ranh giới mô-đun rõ ràng để có thể bổ sung trò chơi mới. Hệ thống ghép nối phải đủ chắc chắn cho việc trình diễn trên bàn; dây nguồn và tín hiệu cần được cố định, không để chân tín hiệu ở trạng thái không xác định. Giao diện phải đọc được trên màn hình 320×240 , phản hồi đầu vào trong quá trình truyền khung hình, và tài liệu phải cho phép người khác build, flash và chạy lại project.

CHƯƠNG 2

Cơ sở kỹ thuật và kiến trúc hệ thống

2.1 Nền tảng phần cứng

Vi điều khiển trung tâm là STM32F429ZIT6. Dòng STM32F429 cung cấp lõi Cortex-M4 với FPU, nhiều ngoại vi truyền thông và USB OTG; chi tiết thanh ghi, kiến trúc bus và ngoại vi được mô tả trong RM0090 [1, 2]. Project cấu hình HSE 8 MHz và PLL để chạy lõi ở 168 MHz. Mức này thấp hơn giới hạn tối đa của linh kiện nhưng cần vì bắt buộc cấu hình xung cấp cho USB OTG ở 48MHz.

Màn hình ILI9341 được điều khiển bằng SPI5 ở chế độ master. Dữ liệu điểm ảnh dùng RGB565, nghĩa là mỗi điểm ảnh chiếm 16 bit.

Bảng 2.1. Cấu hình ngoại vi chính

Khối	Cấu hình	Vai trò
STM32F429ZIT6	HSE 8 MHz, SYSCLK 168 MHz	Chạy vòng lặp, luật game và dựng hình
ILI9341	320 × 240, RGB565; SPI5 42 Mbit/s; PF7/PF9	Hiển thị menu, Stack 3D và Tetris
USART2 RX	921600 baud; PA3; DMA1 Stream 5 vòng	Nhận dữ liệu điều khiển từ ESP32 vào STM32
Nút người dùng	PA0, EXTI cạnh lên, debounce 50 ms	Phát hành động chọn
USB OTG FS	PC4 điều khiển nguồn VBUS	Cấp nguồn cho ESP32; không tham gia truyền dữ liệu game

Các chân điều khiển LCD được khai báo trong driver: CS tại PC2, D/C tại PD13 và Reset tại PD12. ESP32 dùng UART2 TX trên chân GPIO17 để nối tới PA3 (USART2 RX) của STM32; GPIO16 được cấu hình RX trên ESP32 nhưng luồng điều khiển của phiên bản hiện tại chỉ đi một chiều. Hai board phải nối chung mass. SPI5 TX sử dụng DMA2 Stream 4 khi gửi các khối điểm ảnh tới LCD.

2.2 Đường điều khiển không dây

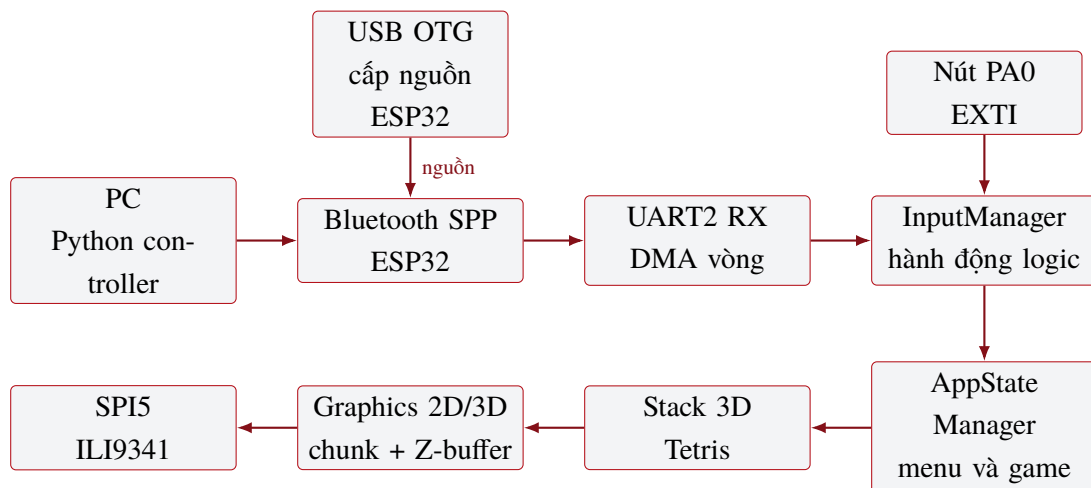
Máy tính chạy `Others/bluetooth.py` để đọc phím tức thời và gửi byte qua socket RF-COMM. Script tự tìm thiết bị ESP32_Bridge_Logger trong Windows Registry, sau đó kết nối Serial Port Profile trên kênh 1. ESP32 dùng `BluetoothSerial` để nhận các byte Blue-

tooth Classic và chuyển nguyên trạng sang STM32 qua UART2 ở 921600 baud. UART2 là đường dữ liệu một chiều từ ESP32 đến STM32 trong hệ thống hiện tại.

Trên STM32, module espuart khởi động DMA nhận vòng 256 byte. Hàm polling so sánh chỉ số đọc với vị trí ghi hiện tại của DMA, lấy từng byte và gọi callback. Callback chuyển ký tự sang `InputAction_t`; nhờ đó, game không phụ thuộc vào nguồn nhập cụ thể. Đường truyền hiện tại là Bluetooth SPP - ESP32 - UART2, không phải I2C.

2.3 Kiến trúc tổng thể

Hình 2.1 mô tả luồng dữ liệu và nguồn của nguyên mẫu. Cổng USB OTG chỉ được dùng để cấp nguồn cho ESP32, không phải một đường nhập liệu. Dữ liệu điều khiển đi từ PC qua Bluetooth SPP đến ESP32, sau đó được ESP32 chuyển sang STM32 qua UART2 RX và DMA vòng.



Hình 2.1. Kiến trúc dữ liệu và điều khiển của hệ thống

2.4 Tổ chức phần mềm

Phần ứng dụng được chia theo trách nhiệm như Bảng 2.2.

Bảng 2.2. Các mô-đun phần mềm chính

Mô-đun	Trách nhiệm	Giao diện chính
main.c	Khởi tạo ngoại vi, lấy Δt , đọc sự kiện nút và gọi vòng lặp ứng dụng	main, callback EXTI/UART
input_manager	Chuẩn hóa ký tự thành hành động logic	InputManager_MapCharacter
app_state_manager	Quản lý menu, game hiện tại, con trỏ và màn hình game over	Init, Update, Render, HandleAction
stack_game	Luật Stack, vật lý mảnh rơi, camera và đối tượng render	Init, Update, HandlePress, Render
tetris_game	Lưới, tetromino, va chạm, xóa dòng, điểm và giao diện	Init, Update, HandleAction, Render
graphics_3d	Toán vector/ma trận, rasterizer, Z-buffer, chunk và overlay	GFX3D_RenderObjects
Driver ILI9341	Gửi lệnh, màu và khối điểm ảnh tới LCD	Init, SetAddress, SendPixelBuffer

2.5 Luồng thực thi

Sau khi HAL và ngoại vi được khởi tạo, chương trình cấu hình nguồn cho ESP32, màn hình, UART2 RX DMA và state manager. Mỗi vòng lặp polling UART2, lấy các lần nhấn PA0 trong vùng bảo vệ ngắt, tính Δt , cập nhật state rồi render toàn màn hình. Kiến trúc không dùng RTOS; mọi phần việc chạy tuần tự trong vòng lặp chính, còn DMA và ngắt chỉ đảm nhiệm tiếp nhận dữ liệu hoặc ghi nhận sự kiện ngắt.

CHƯƠNG 3

Thiết kế và hiện thực

3.1 Quản lý trạng thái ứng dụng

Ứng dụng có ba trạng thái cấp cao: menu chính, Stack 3D và Tetris. Mảng cấu hình `game_screens` lưu tên, hitbox và các con trỏ hàm của từng trò chơi. State manager chỉ gọi qua giao diện `start`, `restart`, `update`, `render`, `handle_play_action` và `is_game_over`. Cách tổ chức này cho phép menu xử lý hai game theo cùng một luồng mà không cần kiểm tra loại game tại nhiều vị trí.

Trong menu, WASD dịch con trỏ 15 điểm ảnh theo mỗi lệnh. Khi nhận hành động chọn, state manager kiểm tra con trỏ có nằm trong hitbox của thẻ Stack hoặc Tetris hay không. Trong trạng thái chơi, lệnh được chuyển cho game hiện tại. Khi game over, con trỏ xuất hiện lại để người dùng chọn RESTART hoặc MENU.

Nút PA0 chỉ tạo `INPUT_ACTION_SELECT`. Vì vị trí con trỏ khởi tạo (160, 120) nằm giữa hai thẻ menu, riêng nút này chưa đủ để chọn game từ lúc khởi động. Người dùng cần nguồn lệnh có các hướng di chuyển, hiện là cầu nối Bluetooth.

3.2 Chuẩn hóa đầu vào

Kiểu `InputAction_t` gồm sáu giá trị: không hành động, lên, xuống, trái, phải và chọn. Bảng 3.1 cho thấy ánh xạ thực tế trong mã nguồn. Enter và các mã phím mũ tên nhiều byte chưa được xử lý.

Bảng 3.1. Ánh xạ đầu vào của phiên bản hiện tại

Ký tự	Hành động	Cách dùng
W/w	Lên	Di chuyển con trỏ hoặc xoay tetromino
S/s	Xuống	Di chuyển con trỏ hoặc soft drop
A/a	Trái	Di chuyển con trỏ hoặc khối Tetris
D/d	Phải	Di chuyển con trỏ hoặc khối Tetris
1, Space	Chọn	Chọn menu, đặt khối Stack hoặc hard drop
PA0	Chọn	Tạo cùng hành động chọn sau debounce 50 ms

3.3 Pipeline dựng hình 3D

Bộ dựng hình không dùng thư viện đồ họa ngoài. Mỗi đối tượng gồm một mesh, ma trận World và màu RGB565. Với đỉnh $\mathbf{p}$, tọa độ clip được tính theo

$$\mathbf{p}_{clip} = \mathbf{P}\mathbf{V}\mathbf{W}\mathbf{p}, \quad (3.1)$$

trong đó $\mathbf{W}$ biến đổi mô hình, $\mathbf{V}$ biểu diễn camera và $\mathbf{P}$ là phép chiếu phối cảnh. Sau phép chia cho thành phần w , tọa độ chuẩn hóa được ánh xạ sang màn hình 320×240 . Giá trị $1/w$ được nội suy và lượng tử hóa về 16 bit để so sánh độ sâu.

Với mỗi tam giác, renderer tính diện tích có hướng trong không gian màn hình. Tam giác có diện tích không dương bị loại như mặt quay lưng. Pháp tuyến trong không gian thế giới được tính bằng tích có hướng của hai cạnh. Cường độ sáng là tích vô hướng với vector ánh sáng, sau đó kết hợp thành phần ambient 0,55 để tạo màu phẳng trên mỗi mặt.



Hình 3.1. Các bước dựng một khung hình 3D

Tam giác được sắp theo tung độ rồi chia thành tam giác đáy phẳng hoặc đỉnh phẳng. Mỗi scanline nội suy biên trái, biên phải và độ sâu. Pixel chỉ ghi màu khi giá trị độ sâu mới lớn hơn giá trị đang lưu.

Thuật toán 1 Dựng nhiều đối tượng theo từng chunk

Đầu vào: Danh sách đối tượng, ma trận View, Projection

Đầu ra: Hai dải điểm ảnh được gửi tuần tự tới LCD

- 1: **với** *chunk* $\leftarrow$ 0 đến 1 **thực hiện**
 - 2: Xóa color buffer và Z-buffer của 120 dòng hiện tại
 - 3: Vẽ nền của chunk
 - 4: **với** mỗi đối tượng **thực hiện**
 - 5: Tính $MVP = P \times V \times W$ và chiếu các đỉnh
 - 6: **với** mỗi mặt tam giác giao với chunk **thực hiện**
 - 7: **nếu** mặt hướng về camera **thì**
 - 8: Tính màu theo pháp tuyến và nguồn sáng
 - 9: Raster hóa scanline, chỉ ghi pixel vượt kiểm tra độ sâu
 - 10: **kết thúc nếu**
 - 11: **kết thúc với**
 - 12: **kết thúc với**
 - 13: Vẽ HUD hoặc con trỏ lên color buffer
 - 14: Đổi thứ tự byte RGB565 và truyền chunk qua SPI5
 - 15: **kết thúc với**
-

3.4 Tối ưu bộ nhớ theo chunk

Hai mảng tĩnh *chunk_color_buffer* và *chunk_z_buffer* đều có 320×120 phần tử 16 bit. Tổng dung lượng là

$$M_{chunk} = 2 \times 320 \times 120 \times 2 = 153600 \text{ byte} = 150 \text{ KiB}. \quad (3.2)$$

Nếu lưu cả màu và độ sâu cho toàn màn hình, dung lượng tương ứng là

$$M_{full} = 2 \times 320 \times 240 \times 2 = 307200 \text{ byte} = 300 \text{ KiB}. \quad (3.3)$$

Vùng RAM thông thường dài 192 KiB và CCMRAM 64 KiB. Hai framebuffer toàn màn hình vượt riêng vùng RAM 192 KiB, trong khi phương án chunk giảm một nửa dung lượng. Thiết kế hiện tại vì thế để lại ít biên RAM cho middleware và trạng thái game.

Mỗi chunk được truyền xuống LCD thông qua kênh DMA2 Stream 4 của SPI5 ở chế độ không chặn (HAL_SPI_Transmit_DMA). Trong thời gian chờ DMA hoàn thành việc truyền dữ liệu màn hình, CPU liên tục gọi hàm thăm dò `espuart_poll()` để tiếp nhận và xử lý các sự kiện điều khiển từ ESP32 qua UART2. Cách xen kẽ này giảm thời gian đầu vào phải chờ so với truyền SPI chặn, nhưng project hiện chưa có phép đo độ trễ đầu vào bằng thiết bị chuyên dụng.

3.5 Thiết kế Stack 3D

Trạng thái Stack chứa tối đa 20 khối đã đặt, một khối đang di chuyển và bốn mảnh rơi. Mỗi lượt đổi trực chuyển động giữa X và Z . Vận tốc tăng theo điểm số người chơi:

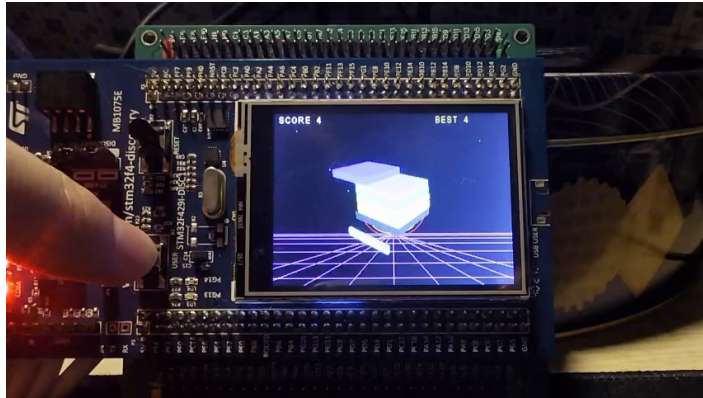
$$v = \min(1.2 + 0.08 \times score, 3.0) \text{ đơn vị/s.} \quad (3.4)$$

Khối hoạt động đi lại trong khoảng ± 2.5 quanh tâm khối đỉnh. Khi người chơi nhấn chọn, thuật toán lấy khoảng giao nhau trên trục hiện tại:

$$overlap = \min(a_{\max}, t_{\max}) - \max(a_{\min}, t_{\min}), \quad (3.5)$$

với a (active) là khối hoạt động và t (top) là khối đỉnh. Nếu $overlap$ không dương, game chuyển sang game over. Ngược lại, phần giao trở thành khối mới; phần thừa trở thành mảnh rơi có gia tốc 4.5 đơn vị/s^2 . Camera nội suy về vị trí khối đỉnh để theo dõi tháp khi chiều cao tăng.

Renderer tạo một cube mesh dùng chung, sau đó thay đổi ma trận tịnh tiến và tỉ lệ cho từng khối. Các đối tượng cùng chia sẻ Z-buffer theo chunk, vì vậy mảnh rơi và các tầng tháp che khuất nhau theo độ sâu thay vì phụ thuộc thứ tự vẽ.



Hình 3.2. Giao diện trò chơi Stack 3D với góc nhìn phối cảnh và bộ đệm độ sâu Z-buffer

3.6 Thiết kế Tetris

Tetris dùng lưới 20×10 phần tử RGB565. Bảy tetromino được mô tả bởi bốn trạng thái xoay, mỗi trạng thái gồm bốn tọa độ tương đối. Trước mọi thao tác, CheckCollision kiểm tra biên trái/phải, đáy và các ô đã khóa. Khi xoay không hợp lệ tại chỗ, chương trình thử dịch một ô sang trái rồi một ô sang phải; đây là wall kick đơn giản trong Tetris cổ điển.

Khi khối chạm đáy hoặc khối đã khóa, bốn ô được ghi vào lưới. Thuật toán quét từ hàng cuối lên trên; với hàng đầy, các hàng phía trên dịch xuống và chỉ số hàng được kiểm tra lại. Điểm cho một đến bốn dòng lần lượt là 40, 100, 300 và 1200, nhân với $level + 1$. Cứ mười dòng, cấp độ tăng và khoảng rơi tự động giảm từ 0,80 giây xuống tối thiểu 0,08 giây.



Hình 3.3. Giao diện trò chơi Tetris hiển thị trên màn hình ILI9341

3.7 Tích hợp hiển thị và giao diện

Menu và Tetris vẽ trực tiếp vào color buffer theo chunk. Stack 3D dùng overlay callback để ghép điểm số, best score và hộp game over sau bước rasterization. Sau khi hoàn thành chunk, hàm gửi buffer đổi byte của từng giá trị RGB565 để byte cao được truyền trước, đặt vùng địa chỉ LCD rồi gửi dữ liệu qua SPI5.

CHƯƠNG 4

Cài đặt, kết quả và đánh giá

4.1 Môi trường phát triển và cách cài đặt

Mã nguồn project được lưu tại <https://github.com/nqd2/stm32-simple-gameboy>. File cấu hình `stm3d.ioc` được tạo cho STM32F429ZITx bằng STM32CubeMX 6.18.0; toolchain đích là STM32CubeIDE. Firmware ESP32 sử dụng Arduino core for ESP32 và thư viện `BluetoothSerial`. Chương trình điều khiển trên PC sử dụng Python 3. Các version không được khóa trong repository phải được kiểm tra lại trên máy build nếu cần tái tạo chính xác môi trường.

Quy trình cài đặt gồm:

1. Clone repository và import project `stm3d` vào STM32CubeIDE bằng chức năng *Existing Projects into Workspace*;
2. Mở `stm3d.ioc`, kiểm tra target STM32F429ZITx, sinh lại mã nếu thay đổi cấu hình, sau đó build cấu hình Debug;
3. Kết nối ST-LINK, flash firmware lên STM32F429 và nối LCD ILI9341 theo bảng chân trong Chương 2;
4. Build và nạp `Others/esp32sketch/esp32sketch.ino` lên ESP32, nối GPIO17 của ESP32 với PA3 của STM32 và nối chung GND;
5. Pair thiết bị ESP32_Bridge_Logger với PC, cài Python và chạy lệnh `pythonOthers/bluetooth.py` để gửi phím.

4.2 Các mô-đun cài đặt chính

Phần cài đặt được chia thành bốn lớp. Lớp phần cứng gồm HAL, cấu hình clock, GPIO, DMA, SPI5 và USART2. Lớp driver gồm ILI9341, font, UART nhận vòng và ánh xạ đầu vào. Lớp nền tảng gồm đồ họa 2D/3D và trình quản lý trạng thái. Lớp ứng dụng gồm menu, Stack 3D và Tetris. Phía ngoài STM32 có ESP32 làm cầu nối byte Bluetooth–UART và script Python bắt phím trên PC. Ranh giới này giúp logic game chỉ phụ thuộc `InputAction_t` và các hàm render, không phụ thuộc trực tiếp vào Bluetooth hoặc thanh ghi ngoại vi.

4.3 Phân công và mức độ đóng góp

Nhóm thống nhất mức đóng góp tổng là 25% cho mỗi thành viên. Bảng 4.1 tính trên 13 commit kỹ thuật trước commit bổ sung tài liệu báo cáo. Số commit chỉ phản ánh cách chia

lịch sử theo phần việc chính; một commit có thể chứa mã sinh tự động hoặc thay đổi giao thoa giữa nhiều mô-đun nên không được dùng riêng lẻ để suy ra khối lượng công việc.

Bảng 4.1. Phân công và đóng góp của thành viên

Thành viên	GitHub	Nhiệm vụ chính	Commit	Đóng góp
Nguyễn Quý Đức	nqd2	Xây dựng engine đồ họa 2D/3D từ driver ILI9341; tích hợp SPI DMA và nền tảng hiển thị	5	25%
Bùi Quang Phương	F3ust	Điều khiển Bluetooth PC-ESP32-STM32; chuẩn hóa đầu vào và menu chọn hai game	3	25%
Trần Đăng Sinh	MikoUwU24	Thiết kế, cài đặt và chuẩn bị demo trò chơi Stack 3D	2	25%
Nguyễn Hồng Dương	BlackHoleHiHi	Thiết kế và cài đặt trò chơi Tetris	3	25%

Email dùng để nhận diện commit lần lượt là `nguyenquyduc_hsgs20@hus.edu.vn`, `hd3876536@gmail.com`, `ctan4991@gmail.com` và `nguyenhongduong01092005@gmail.com`.

4.4 Kết quả và bằng chứng demo

Nguyên mẫu đã hiển thị được menu và hai trò chơi trên LCD ILI9341. Stack 3D có chuyển động luân phiên theo hai trục, cắt phần thừa, mảnh rơi, điểm số và camera theo tháp. Tetris có bảy tetromino, xoay, dịch chuyển, soft drop, hard drop, xóa dòng, điểm và cấp độ. Hai game dùng chung menu, đầu vào và luồng game over/restart.

Bằng chứng demo được lưu cùng repository để link còn hiệu lực trong suốt học kỳ:

- Video Stack 3D: [docs/media/stack-3d-demo.mp4](#);
- Ảnh Stack 3D: [stack-3d-demo-cover.png](#);
- Ảnh Tetris: [tetris-demo-cover.png](#).

4.5 Đánh giá

Bảng 4.2. Đối chiếu kết quả với yêu cầu

Yêu cầu	Trạng thái	Bằng chứng/giới hạn
Menu và hai trò chơi	Đạt	Mã nguồn, ảnh và video demo trong repository
Đồ họa 2D/3D trên ILI9341	Đạt	Renderer, Z-buffer theo chunk và SPI5 DMA
Điều khiển không dây	Đạt	PC–Bluetooth SPP–ESP32–UART2 DMA
Phản hồi thời gian thực	Đạt một phần	Có polling UART trong lúc SPI DMA; chưa đo định lượng input latency và FPS
Độ hoàn thiện cơ khí	Đạt mức nguyên mẫu	Mạch ghép nối phục vụ demo; chưa có PCB/vỏ bảo vệ riêng
Khả năng mở rộng	Đạt	Giao diện game và InputAction_t tách khỏi nguồn nhập cụ thể

Ưu điểm chính là kiến trúc mô-đun, renderer không phụ thuộc thư viện đồ họa ngoài, tiết kiệm RAM bằng chunk và hỗ trợ hai nguồn điều khiển. Hạn chế gồm chưa đo FPS/input latency bằng thiết bị ngoài, chỉ có một nút vật lý cho hành động chọn, giao tiếp Bluetooth phụ thuộc ESP32 và PC, chưa có âm thanh, PCB hoặc vỏ hoàn thiện.

4.6 AI-assisted disclaimer

Nhóm **có sử dụng** công cụ AI để hỗ trợ rà soát cấu trúc repository, đối chiếu báo cáo với mã nguồn, biên tập LaTeX và phát hiện mô tả không còn đúng sau khi project chuyển từ I2C sang UART2. Các prompt chính yêu cầu: phân tích yêu cầu báo cáo cuối kỳ; xác định module phần cứng/phần mềm từ repository; lập bảng phân công và commit; kiểm tra tính nhất quán của thông số SPI, UART, DMA; và đề xuất cách trình bày kết quả, giới hạn. Nhóm chịu trách nhiệm kiểm tra lại nội dung kỹ thuật, lựa chọn thiết kế và kết quả cuối cùng; AI không thay thế việc kiểm thử trên phần cứng.

CHƯƠNG 5

Kết luận và hướng phát triển

5.1 Kết quả đạt được

Đề tài đã xây dựng một nền tảng trò chơi nhúng có cấu trúc mô-đun trên STM32F429. Mã nguồn bao gồm vòng lặp ứng dụng, state manager, lớp chuẩn hóa đầu vào, driver LCD, thư viện đồ họa 2D/3D, Stack 3D, Tetris và cầu nối Bluetooth qua ESP32. Hai trò chơi dùng cùng cơ chế điều hướng menu và màn hình kết thúc, nhưng có mô hình xử lý khác nhau: Stack làm việc với khối hình trong không gian ba chiều, còn Tetris làm việc với lưới hai chiều.

Bộ dựng hình phần mềm là kết quả kỹ thuật nổi bật. Pipeline có biến đổi ma trận, chiếu phối cảnh, loại mặt khuất, chiếu sáng phẳng, rasterization scanline và Z-buffer. Việc xử lý theo hai chunk 120 dòng giảm phần bộ nhớ màu và độ sâu từ 300 KiB xuống 150 KiB, đưa bài toán vào vùng có thể triển khai trên cấu hình linker hiện tại. Overlay theo chunk còn cho phép ghép HUD mà không cần framebuffer bổ sung.

Video và các hình ảnh thực nghiệm trong repository ghi lại nguyên mẫu Stack 3D và Tetris chạy trên phần cứng, hiển thị quá trình chơi, điểm số và trạng thái kết thúc trò chơi.

5.2 Đóng góp của đề tài

Các đóng góp có thể đối chiếu gồm:

- Một kiến trúc ứng dụng thống nhất cho menu và nhiều trò chơi trên vi điều khiển không dùng RTOS;
- Một rasterizer 3D bằng C hỗ trợ nhiều đối tượng, Z-buffer, flat shading và overlay theo chunk;
- Luật Stack 3D có tính giao khối, phần thừa rơi, tốc độ tăng và camera bám tháp;
- Luật Tetris gồm bảy tetromino, xoay có dịch ngang đơn giản, xóa dòng, điểm, cấp độ và preview;
- Đường điều khiển PC - Bluetooth SPP - ESP32 - UART2 DMA được tách khỏi logic game bằng `InputAction_t`.

5.3 Hạn chế và hướng phát triển

Phiên bản hiện tại chưa đo FPS và độ trễ, chưa có âm thanh, bộ điều khiển vật lý đầy đủ, PCB và vỏ. Hướng phát triển là đo hiệu năng, tối ưu rasterizer và truyền LCD, bổ sung âm thanh và lưu điểm, đồng thời chuẩn hóa giao diện đăng ký game.

Tài liệu tham khảo

- [1] STMicroelectronics. *STM32F427xx/STM32F429xx Datasheet, DS9405 Rev. 13*. 2026. Truy cập ngày 18/07/2026 tại <https://www.st.com/resource/en/datasheet/stm32f429zi.pdf>.
- [2] STMicroelectronics. *RM0090: STM32F405/415, STM32F407/417, STM32F427/437 and STM32F429/439 Reference Manual*. Truy cập ngày 18/07/2026 tại https://www.st.com/resource/en/reference_manual/dm00031020.pdf.
- [3] STMicroelectronics. *STM32CubeF4 MCU Firmware Package*. Truy cập ngày 18/07/2026 tại <https://github.com/STMicroelectronics/STM32CubeF4>.
- [4] Espressif Systems. *Bluetooth - Arduino ESP32 Documentation*. Truy cập ngày 18/07/2026 tại <https://docs.espressif.com/projects/arduino-esp32/en/latest/api/bluetooth.html>.
- [5] Nhóm sinh viên thực hiện. *STM32 Retro Play: Stack 3D & Tetris*. Mã nguồn và dữ liệu demo tại <https://github.com/nqd2/stm32-simple-gameboy>.